Patch-Aware KV Cache Compression for Byte-Level Language Models: A Controlled Study of Attention-Guided, Entropy-Guided and Recency Eviction

Aditya Rayaprolu, Kshitij Gupta, Dhwaj Kishore, Arnab Hazra, Riyazuddin Khan

CS F429 Natural Language Processing, BITS Pilani Hyderabad Campus, Group 12

Abstract

Byte-level language models eliminate the tokenizer and thereby avoid its characteristic failures on rare words, orthographic noise, source code and non-Latin scripts. They incur a corresponding cost in sequence length: a text is approximately four times longer in bytes than in byte-pair-encoding (BPE) tokens, and the key-value (KV) cache maintained during autoregressive decoding grows in proportion. We construct a 13.3M-parameter model of the Byte Latent Transformer (BLT) type, which retains one cache entry per patch of bytes rather than per byte, and examine the extent to which its cache can be compressed. The evaluation protocol comprises three random seeds per setting, paired bootstrap confidence intervals, random and recency baselines, a held-out test split and an analytic account of computation. Four results are positive. (i) Eviction combined with 4-bit quantization retains 7.4% of a WikiText cache at a cost of +0.0012 bits per byte (BPB). (ii) An online SnapKV-style policy, which ranks patches by the model's own attention, outperforms recency in 24 of 24 comparisons. (iii) A fixed-size cache sustains decoding of documents four times longer than the training context, and of 20 PG-19 books, with lower loss than context resetting, constant measured memory (0.59 MB against 136.5 MB) and constant time per byte. (iv) The BLT requires 2.8 times fewer floating-point operations per byte than a byte-level Transformer of equal width. We also evaluate a natural hypothesis: that the patches of highest next-byte entropy, the quantity the BLT already computes to place patch boundaries, are the most valuable to retain. The hypothesis is refuted. Entropy-ranked retention is worse than random selection in 68 of 72 comparisons and worse than recency in 70, and the deficit increases approximately tenfold under tenfold longer training. An error analysis attributes this to the selection of patches of only one or two bytes at the expense of those containing complete words. We state what is novel, what did not succeed, and what the present scale of experiment supports.

1 Introduction

Task. We study language modelling, that is, the prediction of each symbol of a text from the symbols that precede it. Models are evaluated in bits per byte (BPB) on held-out documents, lower values being better. Production systems generally operate on subword tokens produced by BPE [21]. Byte-level models dispense with the tokenizer and are consequently robust to rare words, typographical errors, code and non-English scripts [18, 22, 26, 27]. The cost is sequence length: a byte sequence is approximately four times longer than its BPE counterpart (4.0 bytes per token in our data).

Motivation. During generation, a decoder stores the keys and values of all preceding positions in every layer. This KV cache, rather than the model weights, constrains batch size and context length at deployment. A byte-level Transformer of our width requires 151 MB per sequence at 16,384 bytes, compared with 38 MB for BPE. The BLT [18] applies its large global Transformer to patches of bytes and therefore retains one cache entry per patch. To determine patch boundaries it already computes the entropy of the next-byte distribution, a measure of the model's predictive uncertainty.

Choice of problem. The Part 1 proposal rested on three observations: the KV cache, rather than the weights, limits the number of users that can share a GPU and the context length that can be served; tokenizer-free models quadruple that cache; and the entropy used by the BLT to place patches is discarded after patching, whereas existing eviction methods rely on recency or on run-time attention scores. The problem is practically relevant, small enough to study from scratch on a single GPU, and informative irrespective of the outcome.

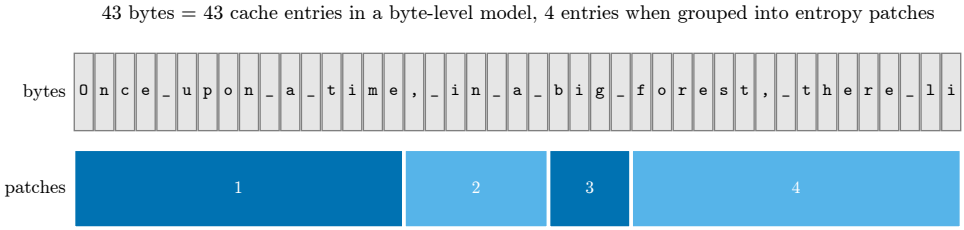


Figure 1. Why patches relieve the cache. The first 43 bytes of a genuine TinyStories document (spaces shown as underscores) and the four entropy patches into which the model gathers them. A byte-level model stores one key-value entry per byte; the BLT-style model stores one per patch.

Research question. Can this entropy additionally determine which patches to retain when the cache is full, or which to store at reduced precision? The signal is available at no extra cost, so a positive answer would yield a free compression criterion. Prior work uses uncertainty to set cache budgets or as one component of a composite score, in subword models (Section 2); to our knowledge, entropy-ranked retention has not been evaluated in byte-level models. We compare it with random, recency and attention-based criteria at equal cache size in bytes. It performs worse than all three, whereas the same experimental protocol yields four positive results (Table 1).

Summary of findings. Consider a passage of 1,000 bytes. A byte-level Transformer stores 1,000 cache entries; the present model groups the bytes into approximately 220 patches and stores 220 entries, about 2 MB. Retaining the patches of highest entropy is a poor criterion: it selects fragments of one or two bytes, discards whole words, and underperforms random selection. Two alternatives succeed. Ranking patches by the model's own attention and storing the retained entries in 4 bits reduces the cache to 7.4% of its original size at a cost of 0.0012 BPB. A fixed-size cache permits decoding of documents four times longer than the training context, in constant memory, with lower loss than resetting the context at each boundary.

2 Literature survey

Tokens and bytes. BPE merges frequent byte pairs into a fixed vocabulary [21]. Token-free models operate on bytes: ByT5 [26] is competitive with subword models and more robust to noise; MegaByte [27] combines a local model within patches with a global model between them; SpaceByte [22] inserts larger blocks after word-boundary bytes. The Byte Latent Transformer [18] forms patches dynamically from the next-byte entropy of a small byte model, applies a large latent Transformer to them, and shows in a FLOP-controlled study up to 8B parameters that patches scale better than tokens. Our model is a simplified BLT with mean pooling in place of cross-attention, no n-gram hash embeddings, and 13.3M parameters.

KV cache eviction. StreamingLLM retains a few initial "sink" positions and a recent window, which allows a model to stream far beyond its training length [25]. H2O retains the "heavy hitters", the positions with the largest accumulated attention [28]. SnapKV selects the positions attended to by the last queries of the prompt [14]. All three operate on subword models. We implement recency with a sink, H2O and a SnapKV-style rule (attention from the last 16 patches) as policies over patch entries.

KV quantization. KIVI quantizes keys per channel and values per token to 2 bits without tuning [15]; KVQuant adds non-uniform and outlier-aware schemes for very long contexts [9]. We use a simpler per-vector absmax quantizer and ask only whether entropy can allocate bit widths.

Uncertainty in cache management. CONF-KV sets the per-step cache budget from the next-token entropy and ranks tokens by attention and recency, with random eviction as a control [13]. InfoKV combines predictive entropy and representation change with attention [10]; counter-causal surprise scores cached tokens by how well later tokens predict them [5]; a learned controller assigns 2 to 16 bits per token from features that include entropy [7]; and Scratchpad Patching adds temporary slots within byte patches according to next-byte uncertainty [29]. We read the first two works in full and the abstracts of the others, so our statement of difference from the latter is provisional. In all of these works entropy is one ingredient or sets a budget; to our knowledge none ranks the patches of a byte-level model by entropy alone against random and recency at equal size.

Relation to widely used models. Widely used models read BPE tokens and address cache growth in the architecture: Llama 3, for example, uses grouped-query attention with 8 key-value heads, a fourfold reduction relative to one KV head per attention head in the 8B model [6]. Eviction and quantization are applied on top of such designs. We study the same bottleneck at a scale that one GPU can train from scratch (13.3M parameters, about 600 times smaller than an 8B model) and for raw bytes, which need about four times as many positions. We make no quality comparison with chat models; what transfers is the measurement protocol, the policies, and the finding that attention-guided rules are effective whereas entropy is not.

Table 1. This work against the previous techniques. "Previous" is the Part 2 pipeline (n-gram and BPE Transformer), the standard recency (StreamingLLM-style) cache, context resets for long documents, and a byte-level Transformer. Evidence in the sections named in the last column.

Aspect	Previous technique	This work	Section
KV cache at 4,096 bytes	Byte-level Transformer 37.7 MB; BPE Transformer 9.4 MB (hypothetical length); BLT full cache 8.4 MB	0.2 MB with 64 patches in 4-bit (+0.0026 BPB on WikiText); 7.4% of the BLT cache with 128 patches in 4-bit for +0.0012 BPB	5.5
Eviction rule	Recency with a sink	Online SnapKV-style: better in 24 of 24 comparisons, by 0.0002 to 0.0077 BPB	5.5
Long documents	Reset the context every 4,096 bytes	Fixed recency cache: −0.0030, −0.0032, −0.0008 BPB (WikiText, 64 patches, three seeds); −0.0086 (20 PG-19 books, 256 patches); measured memory constant, time per byte 0.9 ms against a rise to 2.0 ms	5.5
Compute per byte	Byte-level Transformer, 30.9 MFLOPs (1,024 bytes)	BLT 11.1 MFLOPs, 2.8 times fewer (4.75 times at 4,096 bytes); analytic	5.5
Patching	Fixed stride 4 or 5	Entropy patching is lower in BPB in 3 of 3 same-seed pairs on WikiText validation and test, 67% of patch starts at word starts; gap about one seed range, so likely rather than established	5.2
Modelling quality	Kneser-Ney 4-gram 2.4103; BPE Transformer 1.5200 (WikiText val)	BLT 1.6801 at equal training (behind BPE); 1.29 to 1.31 with ten times more training	5.2
Entropy-guided retention (the proposal)	Random and recency	Not an improvement: worse than random in 68 and recency in 70 of 72 comparisons; penalty grows with training	5.3, 5.6

The classical baseline. Interpolated Kneser-Ney smoothing [2, 11] is the most accomplished classical n-gram method and furnishes a floor that any neural model ought to clear.

3 Dataset

Sources and suitability. TinyStories [4] is a synthetic corpus of short stories generated by GPT-3.5 and GPT-4 (about 890 bytes each), which permits rapid training of small models. WikiText-103 [17] contains long Wikipedia articles (about 18 KB each), suitable for experiments on long-context caches. The two lie at opposite ends of difficulty (our baselines reach 0.69 and 1.52 BPB). The Stack [12] (the Smol sample) adds code, and PG-19 [20] supplies very long documents. The first three corpora serve for training and validation; the Wikipedia test split was used once, at the end.

Limitations and bias. TinyStories is synthetic, narrow and repetitive, and long training overfits it (37 epochs of a 26.7 MB file). WikiText-103 is English and encyclopedic, so its coverage of conversational and non-English text is poor, and the corpora are imbalanced in size (26.7 MB against 526 MB of training text; code is 15% of the mix). The code sample consists of permissively licensed files from which only exact duplicates were removed, so near-duplicates may make validation code slightly easier. PG-19 is literary English of the early twentieth century and earlier, far from the WikiText distribution, and its BPB is correspondingly high. The copy of WikiText training data used for the demonstration is limited to 3,000 articles to keep the download small; all reported runs use the full corpus.

Preprocessing. WikiText articles were reassembled from the raw line export, which lacks article boundaries, and placeholder tokens such as `@-@` were restored. All text is NFC-normalised and exact duplicates are removed. Each split is stored as a single uint8 file of UTF-8 bytes with an offset array; documents are separated by a BOS byte and an attention mask; code is split by content hash so that duplicates never occur in more than one split. All thresholds and hyperparameters were tuned on validation data alone.

4 Method

Model. The network is of the BLT type and has 13,333,632 parameters. A local encoder (1 Transformer layer, width 256, 4 heads, rotary positions, a causal window of 256 bytes) reads the bytes. The encoder states of the bytes within a patch are mean-pooled and projected to width 384. A global Transformer (6 layers, width 384, 6 heads) operates causally over the patches. A local decoder (2 layers, width 256, window 256) predicts each byte from its own encoder state and the global output of the preceding patches. The global keys and values are the cache that we compress: one entry per patch, $2 \cdot 6 \cdot 384 \cdot 2 = 9{,}216$ bytes at fp16. Step-by-step decoding equals the full forward pass to 10^{-5} (tested).

Patching. Entropy patching follows the BLT: a separate byte model of 3.3M parameters (4 layers, width 256, window 512) supplies the next-byte entropy H_i, and a patch begins where H_i exceeds a threshold tuned on validation data to give 4.5 bytes per patch (1.09 nats on TinyStories, 2.17 on WikiText). The decision depends only on earlier bytes (tested). Fixed-stride patching (4 or 5 bytes) serves as the control. Under entropy patching 67.8% (TinyStories) and 66.7% (WikiText) of patch starts fall on word starts, against 19.9% and 16.3% expected by chance.

Table 2. Corpora. Bytes are after normalisation and deduplication. Evaluation sets are fixed and shared by every model.

Corpus	Domain	Size and use
TinyStories	Synthetic children's stories	30,000 train stories (26.7 MB); 21,990 validation; evaluation on the first 500 validation stories (405,231 bytes); no test split
WikiText-103	English Wikipedia articles	29,002 train articles (526.4 MB, 442 duplicates removed); validation 60 articles (1,129,194 bytes); test 62 articles (1,269,744 bytes)
The Stack Smol	Python, Java, JavaScript files	28,229 files, 15% of each training batch in the code-mix models; 200 validation files
PG-19	Pre-1919 books	test split, 100 books (40.5 MB); first 100,000 bytes of 20 books, streaming only

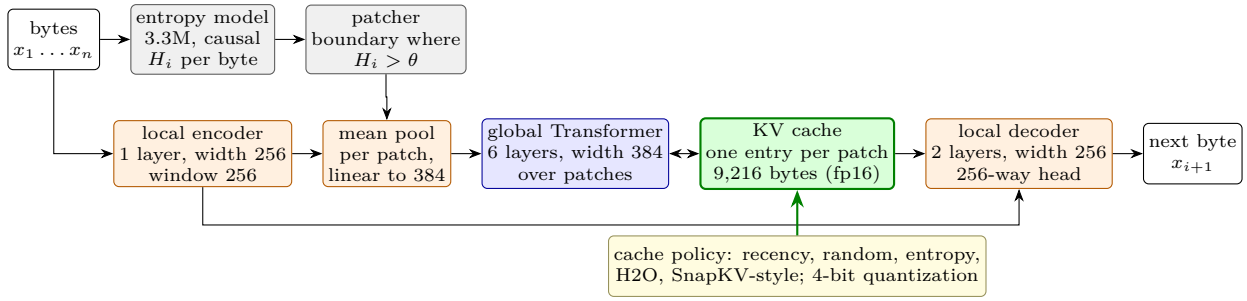


Figure 2. The BLT-style model and where the cache policy acts. Bytes are patched by the entropy model, encoded locally, pooled per patch and processed by a global Transformer whose keys and values are the cache; the local decoder predicts each byte from its own encoder state and the global output of earlier patches. The cache policy decides which patch entries are kept and at which precision.

Cache policies. A policy retains at most B patch entries. The first patch (the sink) and the 8 newest are always retained, and every policy selects among the remainder, so that differences between policies arise from the scoring signal and not from the window. The scores of old patches are as follows.

- *recency*: newest first (as in StreamingLLM);
- *random*: three draws, averaged;
- *keep high entropy* (the hypothesis tested): the mean entropy of the patch, $s_j = |P_j|^{-1} \sum_{i \in P_j} H_i$, the largest values being retained;
- *keep low entropy*: the opposite control;
- *H2O*: attention received so far, summed over queries, heads and layers;
- *SnapKV-style*: attention received from the last 16 patches alone.

Entropy and recency policies are simulated by masks in a single forward pass and are tested to equal step-by-step decoding in which entries are actually removed. H2O and SnapKV-style policies require the evicted cache, so an online forward pass (`forward_online`) runs the global layers patch by patch, selects the entry to discard before the new patch is attended to, and accumulates attention as it is produced (tested for causality and for equality with the full cache at a large budget).

Quantization. Each K and V vector is stored with symmetric absmax scaling at b bits plus one fp16 scale per head. The resulting sizes per entry are 9,216 bytes (fp16), 2,448 (4-bit, 27%) and 1,296 (2-bit, 14%). Tier policies assign a high width (4 bits) to a fraction of the patches and a low width (2 bits) to the rest, selected by recency, at random, or by entropy, and are compared with uniform widths. Eviction and quantization are combined by quantizing every entry that is retained.

Implementation and environment. Models, policies, quantizer and training loop are our own PyTorch code, with rotary positions [23], LayerNorm [1], GELU [8], a 4× MLP and pre-norm blocks [24]; training employs AdamW [16] and PyTorch [19]. No pretrained checkpoint is used; every weight is trained from nothing. Hyperparameters (BLT): AdamW, learning rate 10^{-3}, weight decay 0.01, 200 warm-up steps then cosine decay to 10%, gradient clip 1.0, bf16 autocast, batch 8 of 4,096 bytes, 3,000 steps (30,000 for the long runs), seeds 0 to 2. Entropy model: learning rate $2 \cdot 10^{-3}$, batch 32 of 1,024 bytes, 3,000 steps, clip 1.0. Seed 0 of the prose-only models was trained on a laptop NVIDIA RTX 4060 (8 GB, Python 3.13, PyTorch 2.9) and the remaining runs on a cluster NVIDIA RTX PRO 6000 (95 GB); the demonstrations and the analyses of Section 5.6 run on CPU.

Streaming. For long documents the fixed-size cache processes 16,384-byte chunks (and, for PG-19, 100,000 bytes one byte at a time with the incremental decoding routine), and is compared with a baseline that resets the context every 4,096 bytes, the training length.

5 Experiments

5.1 Setup, metrics and baselines

Training budgets. The principal models are trained on about 98M bytes (3,000 steps). Some WikiText code-mix models are trained for 30,000 steps (about 983M bytes, 1.9 epochs) and at a context of 16,384 bytes. The same checkpoint yields identical scores on the laptop and the cluster GPU.

Table 3. BPB on identical documents, lower is better. TS is TinyStories, WT is WikiText-103. First rows are the Part 2 baselines. BLT rows are seed 0, 3,000 steps, validation; the last row is the mean of the three 30,000-step code-mix seeds (1.2949, 1.2983, 1.3118), which saw ten times more training than the baselines.

Model	TS val	WT val	WT test
KN byte 4-gram	1.8085	2.4103	2.4381
BPE Transformer 13.8M	0.6939	1.5200	1.5309
Entropy model 3.3M	0.8438	1.6983	n/a
BLT, stride 4	0.7726	1.7600	1.7894
BLT, entropy patching	0.7980	1.6801	1.6986
BLT, entropy, 30k steps	n/a	1.2983	n/a

Metric. Bits per byte is the total negative base-2 log-likelihood of a document divided by its length in bytes. It is independent of the tokenizer, so the 4-gram, the BPE Transformer and the byte models are compared on identical text, and it equals the code length per byte that an arithmetic coder driven by the model would achieve [3]. Its limitations are as follows. It measures prediction and not downstream use such as retrieval. Differences between cache policies are small (at most 0.03 BPB), so every comparison is paired by document and reported with a 95% bootstrap interval over documents (2,000 resamples). The Part 2 BPE figure is optimistic by 0.4 to 0.5%, because the first token of each 256-token chunk serves only as context while its bytes are counted (corrected values 0.6978 and 1.5257). Cache size is the mean resident global KV size computed from entry counts and bit widths, and was measured on a GPU for recency alone.

Baselines. The classical baseline is an interpolated Kneser-Ney byte 4-gram; the modern baseline is a 13.8M-parameter BPE Transformer (6 layers, 6 heads, width 384, 8k vocabulary, 256-token context, 3,000 steps). The cache baselines are the full cache, recency with a sink, random selection, H2O, a SnapKV-style rule, and uniform 8, 4, 3 and 2 bits. The Transformer was trained before gradient clipping was introduced (clipping gives 0.6826 on TinyStories), so the modern baseline is slightly handicapped.

5.2 Baselines and base models

Table 3 gives the comparison. All neural models are far ahead of the 4-gram (a gap of 1.0 to 1.1 BPB on TinyStories and 0.65 to 0.9 on WikiText). At equal training the BPE Transformer outperforms our BLT-style model on both sets (0.69 against 0.80 on TinyStories, 1.52 against 1.68 on WikiText); our model is smaller, trained more briefly, and lacks the hashed n-gram embeddings and cross-attention of the original BLT, so we make no claim of superiority over BPE at equal training. With tenfold training (a comparison that favours the BLT, since the Transformer retains 3,000 steps) the WikiText code-mix BLT reaches 1.29 to 1.31, and its cache is the one we compress below.

Table 4. BPB of the seed-0 entropy-patched model under cache budgets (first patch and 8 newest protected). Full cache: 0.7980 (TinyStories), 1.6801 (WikiText). Share of full cache bytes in brackets.

Budget	recency	random	low ent.	high ent.
TinyStories 32 (32%)	0.8078	0.8115	0.8089	0.8231
TinyStories 64 (56%)	0.8016	0.8033	0.8026	0.8109
WikiText 64 (15%)	1.6836	1.6860	1.6901	1.6922
WikiText 128 (28%)	1.6814	1.6833	1.6864	1.6885

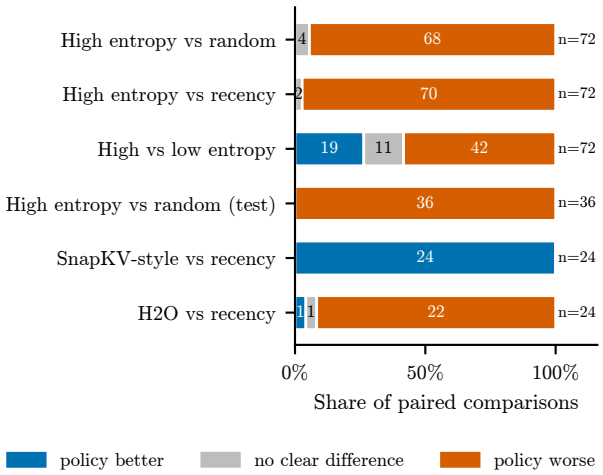


Figure 3. Scoreboard of paired comparisons, counted from the result tables. Keeping high-entropy patches is never better than random or recency; the SnapKV-style rule beats recency in every comparison; H2O mostly loses to recency.

Entropy patching and fixed-stride patching, three seeds each (mean full-cache BPB): on WikiText 1.6958 (entropy) against 1.7347 (stride 4) and 1.7629 (stride 5), entropy patching being lower in all three same-seed pairs and using fewer cache entries per byte than stride 4; on TinyStories 0.8183 against 0.8237 and 0.8521. Seed ranges are 0.056 and 0.028 for entropy patching and 0.135 and 0.044 for stride 4, so the WikiText gap of 0.039 is comparable to a seed range. We therefore regard the advantage as likely but not established. On TinyStories, entropy patching merely ties stride 4. An initial single-seed comparison that placed entropy patching 3.3% behind stride 4 on TinyStories did not survive the additional seeds and was withdrawn.

5.3 Entropy-guided eviction: a negative result

Table 4 and Figure 3 show that entropy-ranked retention is outperformed by random selection and by recency at every budget. The result is robust across all the evidence gathered:

- Over 72 comparisons (9 models in each of 4 settings, 2 budgets), keeping high entropy is worse than recency with the interval above zero in 70 and indistinguishable in 2, and worse than random in 68 and indistinguishable in 4; it is never better than either. The paired difference against random is +0.0003 to +0.0245 BPB.

- It holds in every seed; for fixed-stride models (so it is not caused by the unevenness of entropy patches); with the code mix; on the WikiText test split (all 36 model and budget cells are worse than random and recency); at 16,384 bytes (+0.016 to +0.021 against random at 64 to 512 patches); and for a recency-protected variant (newest half retained, the other half chosen by entropy: worse than recency in 23 of 24 cells).
- Ablations do not change the conclusion (three seeds): the model's own entropy, the first-byte score, the mean score and sink sizes 0 and 4 are worse than random in all seeds on both datasets; the maximum rule is worse in 3 of 3 (WikiText) and 2 of 3 (TinyStories) seeds; only the sum rule on TinyStories is ever better (one seed, −0.0024).
- Keeping *low* entropy is better than keeping high entropy in 42 of 72 comparisons, worse in 19 and tied in 11, so low-entropy retention is not a dependable alternative either.
- A byte-level cache (one entry per byte) loses far more than a patch cache at equal entry counts: at 32 entries recency costs +0.2286 BPB against +0.0133 with patches. Patches are better per byte of cache; the byte-level cache is competitive only with about 4.5 times as many bytes.

5.4 More training makes the penalty larger

A possible objection is that a small, briefly trained model makes little use of its cache. WikiText code-mix models trained for 30,000 steps score 1.2949, 1.2983 and 1.3118 BPB, and keeping high entropy costs +0.0287, +0.0284 and +0.0238 against random at 64 patches (compared with +0.0020 at 3,000 steps), every interval excluding zero. Any eviction also costs more (recency at 64 patches costs 0.018 BPB over the full cache, up from 0.002), and the fixed-stride model shows the same sign. The TinyStories version overfits (training loss 0.14 nats, validation BPB 1.6493) and is not used. A longer context does not change the conclusion either: our first 16,384-byte models were under-trained and suggested that the penalty diminishes, a conclusion withdrawn after tuning (Section 6).

5.5 What works: attention, quantization, bounded memory

Attention-based scoring. Online, the SnapKV-style rule outperforms recency in all 24 comparisons on the 12 entropy-patched models (by 0.0002 to 0.0077 BPB, for example −0.0013 [−0.0018, −0.0009] on WikiText code mix, seed 0, 64 patches), and is never worse than its offline approximation. With offline scoring it also beats recency in 33 of 36 test-split cells (the other three are ties) and, for the 30,000-step models, by 0.0051, 0.0057 and 0.0049 at 64 patches.

Online H2O is worse than recency in 22 of 24 comparisons, since accumulated attention favours old patches that were popular early. The effect is small (for seed 0 on TinyStories at 32 patches, recency costs 0.0098 over the full cache and the SnapKV-style rule recovers 0.0041), but it is the only signal that outperformed recency, and it is attention rather than surprise.

Table 5. Eviction with uniform 4-bit K and V, mean of three seeds, cost over the full fp16 cache in BPB (share of full cache bytes). Full cache: 0.8183 (TinyStories), 1.6958 (WikiText).

Setting	TinyStories	WikiText
SnapKV-style, 64, 4-bit	+0.0049 (15%)	+0.0026 (3.9%)
recency, 64, 4-bit	+0.0072 (15%)	+0.0046 (3.9%)
keep high ent., 64, 4-bit	+0.0224 (15%)	+0.0196 (3.9%)
SnapKV-style, 128, 4-bit	n/a	+0.0012 (7.4%)
recency, 128, 4-bit	n/a	+0.0019 (7.4%)

Quantization and combination. Uniform 4-bit K and V costs 0.0003 to 0.0014 BPB over fp16 across ten models at 27% of the bytes (uniform 3-bit 0.0014 to 0.0077, 2-bit 0.022 to 0.111). Entropy-guided precision tiers are worse than random and recency tiers in all ten models. Eviction combined with 4-bit storage (mean of three seeds) is shown in Table 5: a WikiText cache of 128 patches in 4-bit holds 7.4% of the full cache bytes and costs +0.0012 BPB with SnapKV-style eviction (+0.0019 with recency), whereas keeping high entropy costs +0.0151. No setting that retains high-entropy patches lies on the bytes-against-BPB frontier.

Bounded-memory streaming. Three WikiText code-mix models trained on 4,096-byte chunks decoded 46 validation documents longer than 8,192 bytes under a fixed patch budget. Recency with 64 patches outperforms the chunked full-cache baseline in all three seeds (by 0.0030, 0.0032 and 0.0008 BPB; with 256 patches by 0.0051, 0.0058 and 0.0035), because the reset discards the context at every chunk boundary whereas a small continuous window does not. The SnapKV-style rule is better still at 64 patches (0.0014, 0.0008 and 0.0024 below recency). Random selection, H2O and keeping high entropy are worse than recency at every budget in all seeds (Figure 4, left).

On PG-19, with the 30,000-step model, 256 patches of recency outperform the reset baseline by 0.0086 BPB [0.0079, 0.0093] and the whole-book full cache by 0.145; the whole-book full cache is worse than the reset baseline (by 0.137), indicating that the model makes no use of a context 24 times longer than its training context. Keeping high-entropy patches is +0.093 to +0.122 worse than recency. Retaining old patches implies attending across distances not seen in training, and we did not renumber positions inside the cache as StreamingLLM does, so PG-19 cannot separate a poor entropy signal from poor long-distance attention.

Memory and speed. A bounded cache holds the global cache at 0.59 to 2.36 MB (64 to 256 patches) while a full cache reaches 136.5 MB at 65,536 bytes, and time per byte stays at 0.9 ms against a rise to 2.0 ms (measured on a GPU, one stream, real eviction of recency). Some fixed costs are not reduced: the 786 KB byte-window caches and the entropy model (about 2.1 MB of window cache, 3.3M parameters), which at 64 patches exceed the global cache. At full precision the patch cache is about 0.9 times the cache of a BPE Transformer of the same width, and a byte-level Transformer requires about four times as much.

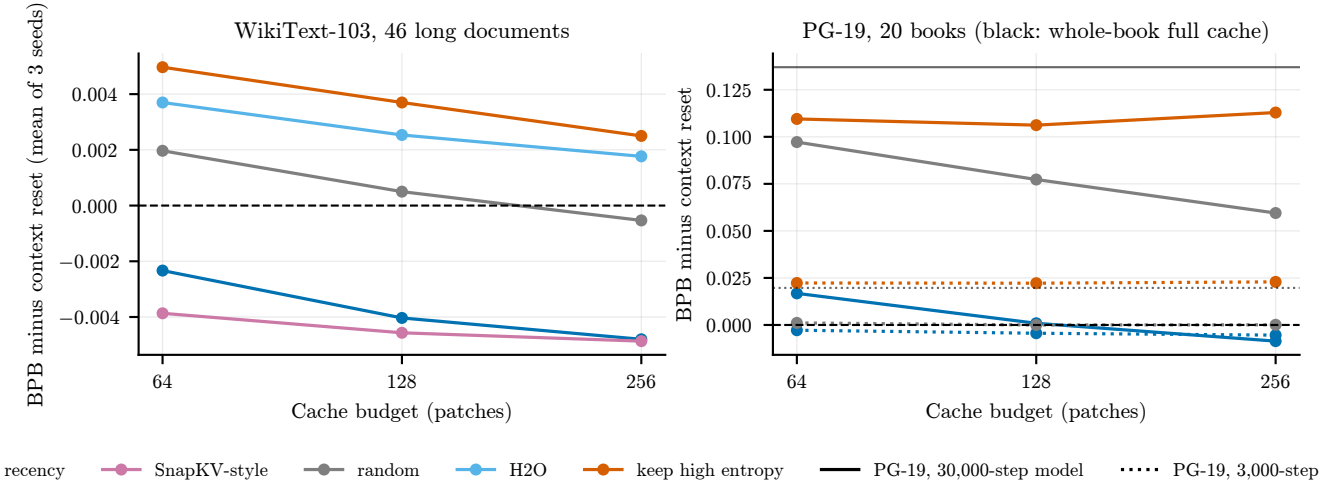


Figure 4. Bounded-memory streaming. Left: WikiText documents longer than 8,192 bytes decoded in 16,384-byte chunks (patch positions four times beyond training); below zero beats resetting the context every 4,096 bytes. Right: 20 PG-19 books of 100,000 bytes decoded one byte at a time (positions about 22,000). Recency is the only policy that is safe on books; keeping high entropy is worst but is confounded by distance (Section 5.5).

Compute per byte. The analytic account is a count of operations, not a measurement. The BLT requires fewer operations than a byte-level Transformer (11.1 against 30.9 MFLOPs per byte at a context of 1,024 bytes) but more than the BPE Transformer (7.4), because its local encoder and decoder run on every byte; when patches are produced by entropy it also requires the separate entropy model (8.5 MFLOPs per byte), giving a total of 19.6. A 64-patch cache reduces only the global attention term (0.3 MFLOPs at 1,024 bytes, 1.7 at 4,096). The advantage of the BLT here is therefore cache memory, not FLOPs.

Retrieval. In a planted-name test (a random name repeated after 400 to 3,000 filler bytes) the models score the same with and without the earlier mention, even at 30,000 steps, so that no policy can be shown to help and nothing about retrieval may be concluded. The entropy rule does retain the name's patches more often than random (45% against 31% at 64 patches and 400 bytes), but the model profits nothing from them.

5.6 Error analysis

Commitments of the proposal. The proposal committed to a breakdown of BPB by entropy, a comparison of code with prose, a retrieval probe, and an examination of generations in which the compressed and full models diverge. All four are addressed: Table 6; the code-domain evaluation (prose-only models score 7.2 to 7.6 BPB on code, the 15% code mix 1.75, and entropy retention is still worse than random there, +0.0036); the planted-name test (uninformative); and Example 4 below. We scored 100 TinyStories documents (74,760 bytes, budget 32 patches) and 30 WikiText documents (122,880 bytes, 64 patches) with the seed-0 models, on a CPU. The documents are the first to pass a length filter; the tables describe one seed and are not tests.

Table 6. Extra loss over the full cache (BPB) by entropy tercile of the patch holding the byte; share of bytes in brackets.

Set	Tercile	recency	high ent.	low ent.
TinyStories	lowest (57%)	+0.0080	+0.0194	+0.0065
	middle (32%)	+0.0104	+0.0272	+0.0130
	highest (11%)	+0.0225	+0.0392	+0.0253
WikiText	lowest (55%)	+0.0023	+0.0152	+0.0035
	middle (31%)	+0.0037	+0.0146	+0.0165
	highest (14%)	+0.0062	+0.0190	+0.0274

Quantitative. Table 6 apportions the extra loss by the entropy tercile of the patch that holds the byte. Every rule loses most on the high-entropy third, where the full-cache loss is also greatest, but keeping high-entropy patches does not protect those bytes (+0.0392 against +0.0225 for recency on TinyStories; +0.0190 against +0.0062 on WikiText), and it also penalizes the easy bytes (+0.0194 against +0.0080 and +0.0152 against +0.0023 on the lowest third, which holds 57% and 55% of the bytes).

The entropy rule retains almost nothing but high-entropy patches (44.6% and 18.1% of the highest third survive, none of the lowest), and they are short: the mean kept patch is 1.15 and 1.13 bytes long and 98.0% and 99.7% of them have one or two bytes, against 4.46 and 4.47 bytes for recency.

This is a confound of length: a surprising byte often forms a patch of its own, so the mean entropy of a short patch tends to be high. It does not explain the whole result, since the sum rule, which favours long patches, ties random on TinyStories and loses on WikiText (Section 5.3).

The loss is concentrated: the largest 1% of bytes carry 30.4% (TinyStories) and 18.0% (WikiText) of the extra loss against recency, and entropy is worse than recency on 40.7% and 51.0% of bytes and better on 33.1% and 42.4%.

Why? Attention. We measured the Spearman correlation between the entropy of an old patch (at least 8 patches back) and the attention it receives. It is negative in the middle and upper layers of most models: in all six layers of all three TinyStories seeds (-0.04 to -0.55), and in layers 2 to 5 of the three prose-only WikiText seeds (-0.007 to -0.39). In the seed-0 TinyStories model the highest-entropy third of old patches receives about half the attention of the lowest third in layers 1 and 2. The model therefore appears to read predictable patches more than surprising ones, and surprising bytes are handled by the local windows. This is a correlation, not an intervention, and the WikiText code-mix seed 0 model shows the opposite sign in most layers while its eviction result keeps the same sign, so the account is incomplete.

Qualitative. In the examples | marks a patch boundary; all are genuine output.

Example 1, TinyStories document 0 (1,201 bytes, 4.0 bytes per patch). Boundaries fall before surprising letters and split names: `R|o|xy.  |Ro|x|y |loved to |c|limb.`, while predictable words remain whole patches (`She |had |never |seen |anything |`). At the last patch the caches (budget 32) hold, for recency, `played |together |all the |time. |And |Ro|x|y |l|earned that |...` (BPB 0.744, +0.010 over full); for keep high entropy, `|s|f| |B|s|s|Ro| |b|f|b|f|a|s|a|i|...` (0.751, +0.017), a sequence of one-letter fragments that keeps `Ro` but not `Roxy`; for keep low entropy, `Once upon a time|, in a |forest, there li|...|never |seen |ried |again, |...` (0.743, +0.009), whole words from the opening of the story.

Example 2, WikiText document 0 (4,096 bytes, budget 64). The entropy cache at the last patch is `o|Ho|E|A|O|S|A|a|6| |a |e|5 |1|e|e|1|...`: capitals, digits and vowels, and no complete word. BPB is 1.752 (+0.033) against 1.725 for recency (+0.006), whose cache holds the section on distribution (`Norway |to |the |A|zores |and |Mo|r|occo`).

Example 3, where entropy helps. In TinyStories document 13 the byte `S` in `All the animals cheered for [S]am` costs 4.04 bits with the full cache, 7.35 with recency and 3.46 with keep high entropy. The name Sam occurs several times earlier in the story. We did not verify whether the entropy cache held a patch containing the name. Such cases are outweighed on average.

Example 4, generations that diverge. Continuing the same prompt greedily with the full cache and with 32 (TinyStories) or 64 (WikiText) patches, recency departs from the full-cache text at byte 17 on TinyStories document 0 (random and keep low entropy at byte 10, keep high entropy at byte 25) and stays with it until byte 65 on WikiText document 1, where random and keep high entropy diverge at the first byte. Greedy text from models this small is repetitive, so this shows where the cache alters the prediction and is not a measure of quality.

Mitigation. Keeping high entropy retains short fragments that carry little context and discards the word-sized patches that the decoder uses. Three remedies were tested. Protecting a recency window and allocating half of the slots by entropy reduces the penalty but does not remove it (worse than recency in all 3 seeds at both budgets on TinyStories); ranking old patches by attention (SnapKV-style) outperforms recency in 24 of 24 comparisons; and quantizing every entry to 4 bits instead of evicting costs at most 0.0014 BPB.

6 Challenges and how we handled them

Table 7. Difficulties, our responses, and what became of them; four required correcting a figure or retiring a claim (last column).

Challenge	What we did	Outcome
Part 2 BPB counted context-only tokens	Kept the reported numbers, disclosed the correction (0.6978 and 1.5257), scored every byte in Part 3	Corrected: the BPE figure was 0.4 to 0.5% optimistic
Part 2 groundwork: WikiText without article boundaries; baseline trained without gradient clipping; blocked cluster access	Articles rebuilt; checkpoints kept (clipping gives 0.6826 against 0.6939); a half-GPU request found to be accepted	Clean documents; slightly weaker baseline, stated; Part 3 trained on the cluster
One job at a time, a 2-day limit	Checkpoint, handle the pre-limit signal, requeue, resume	Verified under one job id (signal at step 1,864, finish at 24,000)
Gated code data; prose models score 7 to 8 BPB on code	A team member's login built the set; models retrained with a 15% code mix	Conclusion unchanged; the mix costs about 0.05 (WikiText) and 0.09 (TinyStories) BPB
Entropy-model BPB below plan; patches must match in training and evaluation	Causality tests, document masks from BOS positions, boundaries from a cached per-byte entropy array	No leak; boundaries consistent to 10^{-3}
A CUDA out-of-memory left a half-written entropy cache	Caught from the log, cache rebuilt, queue stops if the cache step fails	No training on corrupt data
Seed spread as large as some gaps (0.056 to 0.135 BPB)	Three seeds for every patching scheme, a stride-5 control, paired bootstrap by document	Retired: a single-seed 3.3% claim; the eviction sign holds in every seed
Cache byte shares halved by a counting error	Found when two tables disagreed (1,669 KB against 839 KB); fixed, every evaluation rerun	Corrected: BPB unaffected, shares restated
First 16,384-byte models under-trained (1.89 BPB)	Tuned learning rate and batch size on validation (1.5843)	Retired: the penalty does not fade with context; context length and training amount are not separated
TinyStories overfits at 30,000 steps; the PG-19 loader is unsupported	WikiText scale-up as the evidence; a community mirror of the same PG-19 split	TinyStories rows for completeness only; 20 books streamed byte by byte
Offline attention scores approximate; models do not retrieve	Online implementation built and tested; retention rates reported, no retrieval conclusion	Online SnapKV-style better than recency in 24 of 24; claims confined to prediction quality
The central hypothesis was refuted	Reported as measured; tested other scores, own entropy, sink size, a byte-level cache, more training, longer context and a protected recency window; added attention rules; analysed what entropy keeps	A reproducible negative result with a mechanism, beside working compression

7 Scope of the evidence

- *Scale and task.* The models have 13.3M parameters and mostly 3,000 training steps (30,000-step models: same sign, larger effects). They do not retrieve named facts (a planted-name test could not tell any policy from another), so claims concern prediction quality.
- *Data and measurement.* TinyStories has no test split; PG-19 rests on 20 books scored by models trained on other text; cache sizes are computed from entry counts (measured on a GPU for recency only); quantization is simulated; FLOPs are analytic.
- *Seeds.* The 30,000-step fixed-stride and 16,384-byte models have one seed apiece; streaming uses three seeds of one setting.
- *Mechanism.* The accounts of why entropy fails (attention, fragment length) are correlational; no intervention separates them.

8 Conclusion

Novel. A controlled evaluation of entropy-ranked cache retention on a byte-level patch model (multiple seeds, paired intervals, a test split); an online, tested implementation of attention policies over patch caches; and measured evidence that a fixed cache streams documents several times longer than the training context with lower loss than a context reset.

Improved. Eviction with 4-bit quantization retains 7.4% of the cache for +0.0012 to +0.0019 BPB; SnapKV-style eviction outperforms recency in 24 of 24 comparisons; memory and time per byte remain constant under a bounded cache; and the BLT requires 2.8 times fewer FLOPs per byte than a byte-level Transformer.

Negative. The entropy that places patch boundaries is not a useful criterion for selecting patches to retain, and longer training increases its penalty; it retains mainly fragments of one and two bytes. The model's own attention is the signal that does select patches usefully, and with it the cache of a byte-level model can be made small and kept bounded as the text grows (verified to 100,000 bytes).

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech and Language*, 13(4), 1999.

[3] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. *arXiv preprint arXiv:2309.10668*, 2023.

[4] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

[5] Stephen Gould and Anton van den Hengel. Back from the future: Key-value cache management by counter-causal surprise. *arXiv preprint arXiv:2607.27600*, 2026.

[6] Aaron Grattafiori, Abhimanyu Dubey, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[7] Sayed Pedram Haeri Boroujeni, Niloufar Mehrabi, Patrick Woods, Gabriel Hillesheim, and Abolfazl Razi. Don't waste bits! adaptive KV-cache quantization for lightweight on-device LLMs. *arXiv preprint arXiv:2604.04722*, 2026.

[8] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[9] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2401.18079.

[10] Jushi Kai, Zhuiri Xiao, Alexandra Birch, and Zhouhan Lin. Information-aware KV cache compression for long reasoning. *arXiv preprint arXiv:2606.26875*, 2026.

[11] Reinhard Kneser and Hermann Ney. Improved backing-off for M-gram language modeling. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 1995.

[12] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

[13] Yubo Li and Yidi Miao. CONF-KV: Confidence-aware KV cache eviction with mixed-precision storage for long-horizon LLM. *arXiv preprint arXiv:2605.24786*, 2026.

[14] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

[15] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.

[16] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[17] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[18] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.

[19] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. arXiv:1912.01703.

[20] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019.

[21] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Berlin, Germany, 2016. doi: 10.18653/v1/P16-1162.

[22] Kevin Slagle. SpaceByte: Towards deleting tokenization from large language modeling. *arXiv preprint arXiv:2404.14408*, 2024.

[23] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017. arXiv:1706.03762.

[25] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17453.

[26] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 2022. arXiv:2105.13626.

[27] Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. *arXiv preprint arXiv:2305.07185*, 2023.

[28] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arXiv:2306.14048.

[29] Lin Zheng, Vasilisa Bashlovkina, Timothy Dozat, Dan Garrette, Laura Rimell, and Joshua Maynez. Scratchpad patching: Decoupling compute from patch size in byte-level language models. *arXiv preprint arXiv:2605.09630*, 2026.